

Guía Maestra de Estudio: Construcción Plataforma Hack

Materia: Organización de Computadores

1. Lógica del Flujo General (Arquitectura Hack)

La plataforma Hack se basa en la **Arquitectura Harvard**, lo que significa que la memoria de instrucciones (ROM) y la memoria de datos (RAM) están físicamente separadas. El flujo del computador sigue el ciclo clásico **Fetch-Decode-Execute** (Buscar-Decodificar-Ejecutar):

1. **Fetch (Buscar):** El Program Counter (PC) envía la dirección actual a la ROM. La ROM devuelve la instrucción de 16 bits que está en esa dirección.
2. **Decode (Decodificar):** La CPU analiza el bit más significativo (el bit 15) de la instrucción.
3. Si es `0`, es una **Instrucción-A** (carga un valor en el registro A).
4. Si es `1`, es una **Instrucción-C** (realiza un cálculo en la ALU, escribe en memoria/registros, y evalúa saltos).
5. **Execute (Ejecutar):** La ALU realiza operaciones entre el Registro D y el Registro A (o la Memoria M). Dependiendo de los bits de destino, el resultado se guarda. Si se cumple una condición de salto, el PC se actualiza con el valor de A; si no, el PC avanza a la siguiente línea.

Registros Clave: * **D (Data):** Almacena datos temporalmente. Solo interactúa con la ALU. * **A (Address/Data):** Doble propósito. Puede almacenar un valor numérico, o actuar como *puntero* para indicar a qué dirección de memoria RAM (`M`) o ROM (`PC`) nos referimos. * **M (Memory):** Representa el valor en la RAM en la dirección a la que apunta el registro A actualmente (`RAM[A]`).

2. Fase 1: Programación en Ensamblador (Software)

En el ensamblador Hack, no existen estructuras de control nativas como `for` o `while`. Todo se controla mediante **saltos condicionales** (JGT, JEQ, JMP, etc.) comparando siempre el resultado de la ALU con el cero.

2.1. Mult.asm (Multiplicación)

Objetivo: Calcular `RAM[2] = RAM[0] * RAM[1]`. **Lógica:** Como la ALU no multiplica, sumamos `RAM[0]` consigo mismo tantas veces como indique `RAM[1]`. Usamos una variable temporal para no modificar `RAM[1]`.

```
nand2tetris // RAM[2] = RAM[0] * RAM[1] @2 M=0 // Inicializar resultado en 0 @0
D=M @END D;JEQ // Si RAM[0] == 0, saltar al final @1 D=M @END D;JEQ // Si RAM[1]
```

```

== 0, saltar al final @R1_copy M=D // Copiar RAM[1] para usarlo de contador (LOOP)
@R1_copy D=M @END D;JEQ // Si el contador es 0, fin @0 D=M @2 M=M+D // RAM[2] =
RAM[2] + RAM[0] @R1_copy M=M-1 // Decrementar contador @LOOP 0;JMP // Repetir
(END) @END 0;JMP // Ciclo infinito de cierre

```

2.2. Fill.asm (Control de Pantalla y Teclado)

Objetivo: Si se presiona una tecla ($KBD > 0$), pintar la pantalla de negro (-1). Si no, de blanco (0). **Lógica:** Utiliza **I/O Mapeado en Memoria**. La pantalla inicia en `16384` y el teclado está en `24576`. Si se detecta tecla, carga `D=-1` (todos los bits en 1). Un bucle recorre cada registro de pantalla aplicando ese color mediante direccionamiento indirecto (`A=M`).

```

nand2tetris (LOOP) @KBD D=M @WHITE D;JEQ // Si KBD == 0, pintar blanco (BLACK) @-1
D=A @SET_COLOR 0;JMP (WHITE) @0 D=A (SET_COLOR) @color M=D // Guardar el color (-1
o 0) @SCREEN D=A @pixel_addr M=D // Puntero de pantalla (inicia en 16384)
(FILL_LOOP) @pixel_addr D=M @24576 D=D-A @LOOP D;JEQ // Si llegamos al teclado,
reiniciar ciclo principal @color D=M @pixel_addr A=M // Direccionamiento indirecto
al pixel actual M=D // Escribir el color @pixel_addr M=M+1 // Avanzar puntero
@FILL_LOOP 0;JMP

```

2.3. SumN.asm (Sumatoria)

Objetivo: `RAM[1] = 1 + 2 + ... + RAM[0]`. **Lógica:** Implementa un bucle `for(i=1; i<=N; i++)`. Compara `i - N`; si el resultado es mayor a cero (`JGT`), detiene el bucle.

```

nand2tetris @1 M=0 // RAM[1] = 0 @0 D=M @END D;JLE // Si N <= 0, fin @i M=1 // i =
1 (LOOP) @i D=M @0 D=D-M // D = i - N @END D;JGT // Si i > N, salir @i D=M @1
M=M+D // RAM[1] = RAM[1] + i @i M=M+1 // i++ @LOOP 0;JMP (END) @END 0;JMP

```

2.4. CopyBlock.asm (Manipulación de Bloques)

Objetivo: Copiar un arreglo desde `RAM[ORIGEN]` hasta `RAM[DESTINO]` con longitud `N`. **Lógica:** Es el ejercicio supremo de direccionamiento indirecto. Calcula la dirección fuente (`ORIGEN + i`), extrae el dato (`A=D+M; D=M`), luego calcula la dirección destino (`DESTINO + i`) y escribe el dato allí.

```

nand2tetris @2 D=M @END D;JLE // Si N <= 0, terminar @i M=0 // i = 0 (LOOP) @i D=M @2 D=D-M // D = i
- N @END D;JGE // Si i >= N, terminar

```

```

// Leer ORIGEN
@0
D=M
@i
A=D+M // A = ORIGEN + i
D=M // D = RAM[ORIGEN + i]
@temp
M=D // temp = D

// Escribir DESTINO
@1

```

```

D=M
@i
D=D+M      // D = DESTINO + i
@dest_addr
M=D
@temp
D=M
@dest_addr
A=M
M=D      // RAM[DESTINO + i] = temp

@i
M=M+1    // i++
@LOOP
0;JMP

```

(END) @END 0;JMP ``

3. Fase 2: Construcción de Memory (Hardware)

El módulo Memory gestiona tres chips físicos en un espacio de direcciones virtual unificado de 16 bits. Los bits más significativos de la dirección (`address[13..14]`) determinan a qué componente se accede: * `00` y `01` (0 a 16383): Memoria RAM principal. * `10` (16384 a 24575): Pantalla (Screen). * `11` (24576): Teclado (KBD).

Lógica HDL: Usamos un `DMux4Way` para distribuir la señal `load` (decidiendo quién puede escribir), y un `Mux4Way16` para seleccionar qué salida enviar a la CPU al leer.

```
``hdl CHIP Memory { IN in[16], load, address[15]; OUT out[16];
```

```

PARTS:
// Enrutar señal de carga según address[13..14]
DMux4Way(in=load, sel=address[13..14], a=loadRAM1, b=loadRAM2, c=loadScreen, d=loadKBD);
Or(a=loadRAM1, b=loadRAM2, out=loadRAM);

RAM16K(in=in, load=loadRAM, address=address[0..13], out=outRAM);
Screen(in=in, load=loadScreen, address=address[0..12], out=outScreen);
Keyboard(out=outKBD);

// Leer el componente correspondiente
Mux4Way16(a=outRAM, b=outRAM, c=outScreen, d=outKBD, sel=address[13..14], out=out);

```

```
} ``
```

4. Fase 3: Construcción de la CPU (Hardware)

Es el cerebro del sistema. Su trabajo es decodificar la instrucción de 16 bits (`i xx a ccccc ddj jjj`): * `i`: Tipo de instrucción (0=A, 1=C). * `a`: Define si la ALU opera con el Registro A (`a=0`) o con la memoria M (`a=1`). * `c`: Seis bits de control que se conectan directo a los pines de la ALU (`zx, nx, zy, ny, f,`

no). * **d**: Tres bits que activan la carga de los registros A, D y M. * **j**: Tres bits que definen las condiciones de salto evaluando las banderas **zr** (Zero) y **ng** (Negativo) emitidas por la ALU.

```
````hdi CHIP CPU { IN inM[16], instruction[16], reset; OUT outM[16], writeM, addressM[15], pc[15];
```

```
PARTS:
// 1. Decodificador
Not(in=instruction[15], out=isA);
Not(in=isA, out=isC);

// 2. Registro A
And(a=isC, b=instruction[5], out=ALUtoA);
Mux16(a=instruction, b=outALU, sel=ALUtoA, out=inA);
Or(a=isA, b=ALUtoA, out=loadA);
ARegister(in=inA, load=loadA, out=outA, out[0..14]=addressM);

// 3. Mux ALU (¿Opera con A o con M?)
And(a=isC, b=instruction[12], out=selM);
Mux16(a=outA, b=inM, sel=selM, out=inAM);

// 4. Registro D
And(a=isC, b=instruction[4], out=loadD);
DRegister(in=outALU, load=loadD, out=outD);

// 5. ALU
ALU(x=outD, y=inAM,
 zx=instruction[11], nx=instruction[10],
 zy=instruction[9], ny=instruction[8],
 f=instruction[7], no=instruction[6],
 out=outALU, out=outM, zr=zr, ng=ng);

// 6. Señal de escritura de Memoria
And(a=isC, b=instruction[3], out=writeM);

// 7. Lógica de Saltos
And(a=instruction[2], b=ng, out=jLT);
And(a=instruction[1], b=zr, out=jEQ);
Or(a=ng, b=zr, out=ngzr);
Not(in=ngzr, out=pos);
And(a=instruction[0], b=pos, out=jGT);

Or(a=jLT, b=jEQ, out=jLE);
Or(a=jLE, b=jGT, out=jumpCondition);
And(a=isC, b=jumpCondition, out=doJump);
Not(in=doJump, out=noJump);

// 8. Program Counter (PC)
PC(in=outA, load=doJump, inc=noJump, reset=reset, out[0..14]=pc);
```

```
} ````
```

## 5. Fase 4: El Computador Completo (Hardware)

El bloque más alto de la jerarquía. Aquí el hardware toma vida. Conecta la memoria ROM (que contiene

las instrucciones inmutables del programa), la CPU, y la RAM/Periféricos (Memory).

**Lógica:** 1. El `pc` de la CPU le dice a la `ROM32K` qué instrucción leer. 2. La `ROM32K` envía esa instrucción a la `CPU`. 3. La `CPU` interactúa con `Memory` enviando direcciones (`addressM`), datos a escribir (`outM`) y leyendo la RAM actual (`inM`).

```
hdl CHIP Computer { IN reset; PARTS: ROM32K(address=pc, out=instruction);
CPU(inM=inM, instruction=instruction, reset=reset, outM=outM, writeM=writeM,
addressM=addressM, pc=pc); Memory(in=outM, load=writeM, address=addressM,
out=inM); }
```